

Bases de Datos II

Vistas Parte 2



Vistas

- Forman parte del **esquema externo**
 - **presentan una parte de la BD** de interés para grupos de usuarios
 - **ocultando** el resto de la información
- Relación derivada de una o más tablas y/o vistas definidas previamente
- Tabla virtual (habitualmente no materializada)

```
CREATE VIEW nom_vista [(n_col_1, ..., n_col_n)]  
AS expresión_consulta  
[WITH [opción] CHECK OPTION];
```

- **nom_vista**: nombre de la vista
- **n_col_1,..., n_col_n**: nombres de columnas de la vista
- **expresión_consulta**: consulta SQL que define la relación derivada
- **opción**: cascade/local

Actualizaciones en Vistas

- Las vistas no mantienen COPIAS de los datos → cuando se modifica una vista, se están modificando las tablas base

Actualización Tabla Base → Vista

- Al actualizar las tuplas de una tabla → los cambios se reflejan automáticamente sobre las vistas (si no se violan restricciones!)

Actualización Vista → Tabla Base

- Para que resulten automáticamente actualizables deben cumplir ciertas condiciones → Cuándo es posible sin ambigüedades?
 - Si no afecta más de una tabla o, si deriva de más de una tabla, la actualización afecta sólo a una (la que preserva la clave)
 - Si no contiene columnas con información derivada o funciones de grupo o cláusula distinct, subconsultas en el select u operaciones de conjunto
 - Si no causa error afectando atributos que no tienen valores por defecto definidos o que no aceptan nulos o que activan RI asociadas
 - Si verifica la condición, si se especificó *with check option*

Vistas Actualizables

(a partir de 2 o más tablas/vistas)

Según el estándar SQL:1999, la actualización sobre una **vista definida a partir de más de una tabla/vista**^(*)

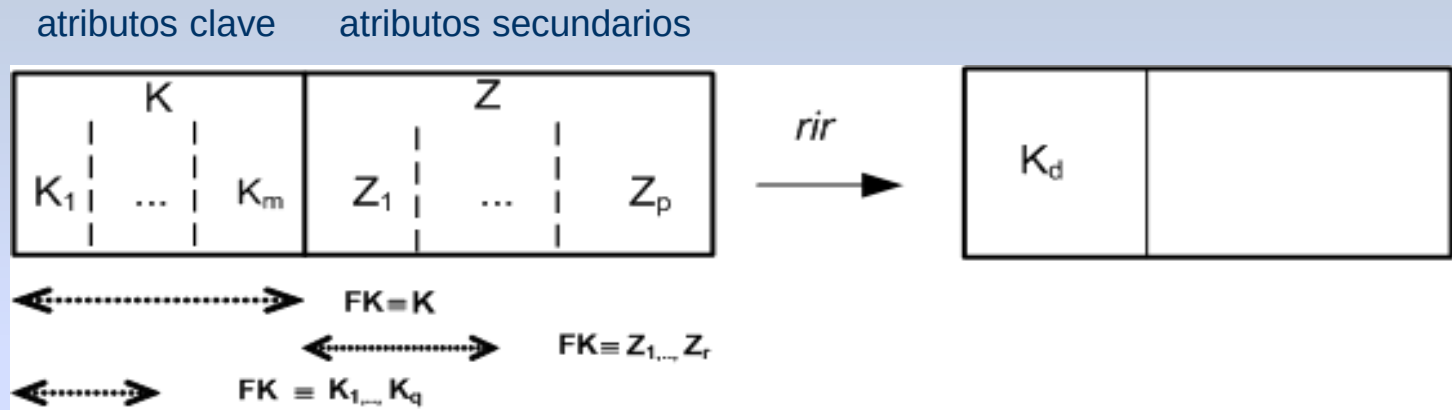
- **sólo puede modificar una de las tablas base**: la que cumpla la propiedad de **preservación de la clave** (la que tiene la misma clave de la vista, y entonces aparece a lo sumo una vez en la vista)
- No debe definirse en base a Unión, Intersección o Diferencia
(Manifiestos para SQL:1999 y versiones posteriores → incluir otras operaciones, por ej. **Intersección**)
- Se denominan **vistas σ - π - $\bowtie$** : se obtienen mediante **condiciones de ensamble sobre los pares FK → PK especificados en las RIRs**
- La actualización no se realizará si se viola alguna restricción definida sobre la relación base a actualizar

(*) Puede ser otra vista, que en este caso debe ser actualizable.

Si $T \rightarrow V1 \rightarrow V2 \rightarrow \dots \rightarrow Vn$: V_i será actualizable si V_{i-1} lo es y así sucesivamente

Comentarios sobre RIRs

Posibles ubicaciones de la FK en relación al esquema de la relación:



- (1) **$FK \equiv K$** $\rightarrow$ procede de la representación de relaciones de tipo-subtipo
- (2) **$FK \equiv Z_1, \dots, Z_r$ (atrib. secundarios); con $FK \cap K = \emptyset$** $\rightarrow$ proviene de la abstracción de relaciones 1:1, N:1 o n -arias con al menos una cardinalidad 1
- (3) **$FK \equiv K_1, \dots, K_q$; con $FK \subset K$** $\rightarrow$ resulta de las relaciones N:N, n -arias en general y las correspondientes a los vínculos entidad fuerte-entidad débil (*rel. identificatorias*)

Vistas Actualizables

(a partir de 2 o más tablas/vistas)

Caso (2): más intuitivo → Ensamble mediante RIR, FK= atrib. secundarios

Ejemplo

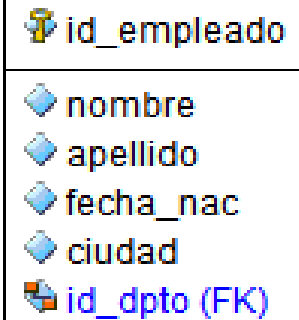
Vista ensamble proveniente de relación N:1

```
CREATE VIEW EMPL_SISTEMAS AS
SELECT E.id_empleado, E.apellido, E.id_dpto, D.nombre
FROM DEPARTAMENTO D, EMPLEADO E
WHERE D.id_dpto = E.id_dpto
      AND D.nombre= 'Sistemas';
```

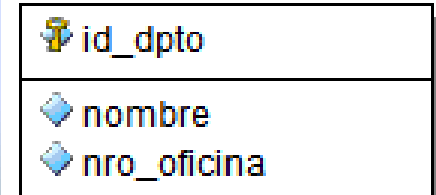
La clave de *EMPL_SISTEMAS* es *E.id_empleado*
(análisis de dependencias funcionales)

Nota: Aunque en la vista hubiera sólo un empleado en cada departamento – y entonces *id_dpto* sería único– igualmente la tabla Departamento NO sería de la cual se preserva la clave! (no depende de los datos sino de la estructura del esquema)

EMPLEADO



DEPARTAMENTO



Vistas Actualizables

(a partir de 2 o más tablas/vistas)

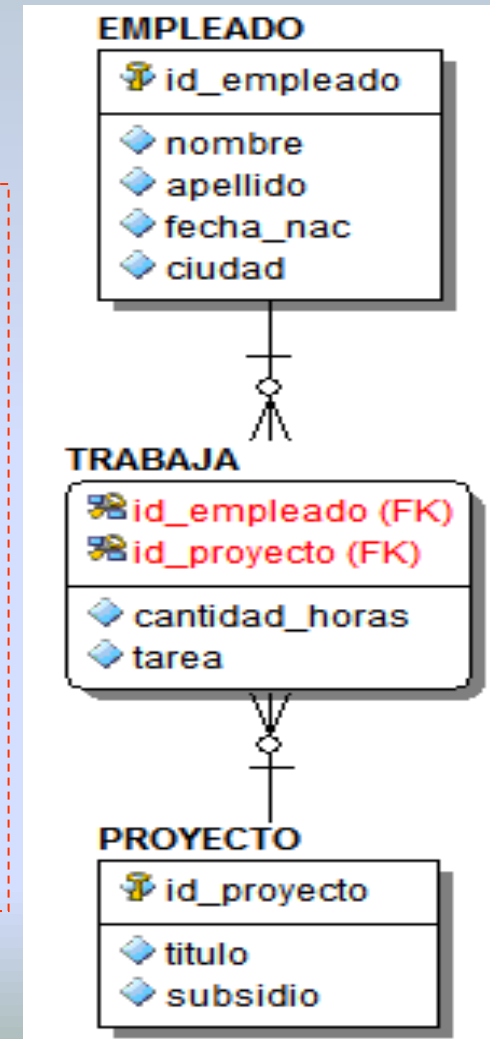
Caso (3): Ensamble mediante RIR, $FK \subset K$ (subconjunto de los atrib. clave)

Ejemplo

Vista ensamble proveniente de relación N:N

```
CREATE VIEW EMPL_PROY AS
SELECT T.id_empleado, T.id_proyecto, T.tarea, E.apellido
FROM TRABAJA T, EMPLEADO E
WHERE T.id_empleado = E.id_empleado
AND E.cantidad_horas < 3000;
```

La clave de *EMPL_PROY* es *T.id_empleado, T.id_proyecto*
(análisis de dependencias funcionales)



Vistas Actualizables

(a partir de 2 o más tablas/vistas)

Caso (1): Ensamble mediante RIR, $FK \equiv K$ (coincide con la clave)

Ejemplo

Vista ensamble correspondiente a relación tipo-subtipo

```
CREATE VIEW ING_TANDIL AS
SELECT I.id_empleado, I.titulo, E.apellido, E.nombre
FROM INGENIERO I, EMPLEADO E
WHERE I.id_empleado = E.id_empleado
      AND E.ciudad = 'Tandil';
```

La clave de *ING_TANDIL* es *I.id_empleado* (tabla subtipo)

Nota: En la vista Empleado e Ingeniero se corresponden con sólo una persona, o sea que *id_empleado* es único en la vista (resultado del ensamble), pero Empleado no sería la tabla de la cual se preserva la clave (porque es esperable que su dominio de definición sea más extenso)

EMPLEADO

 id_empleado

 nombre

 apellido

 fecha_nac

 ciudad

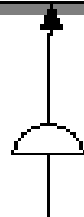
INGENIERO

 id_empleado (FK)

 fecha_graduacion

 titulo

 universidad



Ejemplo

```
create table alumnos(  
  documento char(8),  
  nombre varchar(30),  
  nota decimal(4,2),  
  codigoprofesor int,  
  primary key(documento)  
);
```

```
create table profesores (  
  codigo int auto_increment,  
  nombre varchar(30),  
  primary key(codigo)  
);
```

```
insert into alumnos values('30111111','Ana Algarbe', 5.1, 1);  
insert into alumnos values('30222222','Bernardo Bustamante', 3.2, 1);  
insert into alumnos values('30333333','Carolina Conte',4.5, 1);  
insert into alumnos values('30444444','Diana Dominguez',9.7, 1);  
insert into alumnos values('30555555','Fabian Fuentes',8.5, 2);  
insert into alumnos values('30666666','Gaston Gonzalez',9.70, 2);
```

```
insert into profesores(nombre) values ('Maria Luque');  
insert into profesores(nombre) values ('Jorje Dante');
```

```
create view vista_nota_alumnos_aprobados as
select documento,
       a.nombre as nombrealumno,
       p.nombre as nombrefesor,
       nota,
       codigoprofesor
from alumnos as a
join profesores as p on a.codigoprofesor=p.codigo
where nota>=7;
```

```
select * from vista_nota_alumnos_aprobados;
```

```
-- Mediante la vista insertamos un nuevo alumno calificado por el profesor
-- con código 1
```

```
insert into vista_nota_alumnos_aprobados(documento, nombrealumno, nota, codigoprofesor)
values('99999999','Rodriguez Pablo', 10, 1);
```

```
select * from vista_nota_alumnos_aprobados;
```

```
-- si consultamos la tabla base: alumnos tenemos una nueva fila con el alumno
-- insertado
```

```
select * from alumnos;
```

```
-- modificamos la nota de un alumno aprobado mediante la vista
update vista_nota_alumnos_aprobados set nota=10
where documento='30444444';
```

```
select * from alumnos;
```

Vistas – Actualización vía Triggers

- Recurso que **permite la actualización de vistas** que no son automáticamente actualizables
 - Definir **triggers INSTEAD OF** (opción especial para vistas) para las distintas operaciones requeridas (eventos críticos)
 - Se pueden “interceptar” las operaciones de actualización: el trigger se dispara automáticamente **en lugar de** la sentencia disparadora, en forma “invisible” para el usuario
 - Por defecto, los triggers INSTEAD OF son *for each row*
- Importante: Queda en manos del usuario la responsabilidad de implementar las actualizaciones necesarias y de la manera que las considere “adecuadas”



Vistas – Actualización vía Triggers

Ejemplo

```
CREATE VIEW info_tutores (nro_al, nom_al, id_tutor, nom_tut)
AS SELECT A.nro_al, A.nombre, A.id_tutor, P.nombre
   FROM Alumno A JOIN Profesor P
   ON A.id_tutor = P.id_prof;
```

Alumno (<u>nro_al</u> , nombre, <u>id_tutor</u>) Profesor (<u>id_prof</u> , nombre)

Una **posible semántica para el insert** sobre *info_tutores* podría ser:

(no significa que sea la única, dependerá de los requerimientos)

```
CREATE TRIGGER insert_info_tutores
INSTEAD OF INSERT ON info_tutores
FOR EACH ROW
BEGIN
```

sintaxis SQL
estándar

```
  IF NOT EXISTS (SELECT * FROM Alumno A WHERE A.nro_al= :new.nro_al)
    THEN INSERT INTO Alumno VALUES (:new.nro_al, :new.nom_al, :new.id_tutor);
    ELSE UPDATE Alumno SET id_tutor= :new.id_tutor WHERE nro_al= :new.nro_al;
  END IF;
  IF NOT EXISTS (SELECT * FROM Profesor P WHERE P.id_prof= :new.id_tutor)
    THEN INSERT INTO Profesor VALUES (:new.id_tutor, :new.nom_tut);
    ELSE UPDATE Profesor SET nombre= :new.nom_tut WHERE id_prof= :new.id_tutor;
  END IF;
END;
```

Deberían plantearse triggers similares para las operaciones de UPDATE y DELETE (en PostgreSQL la función podría implementar el comportamiento para todos los eventos)
--

Vistas Actualizables en MySQL

Una vista en MySQL **no es actualizable** si:

- Contiene funciones de agregación
- DISTINCT
- GROUP BY
- HAVING
- Usa UNION o UNION ALL
- Usa subconsultas en el select
- Subconsultas independientes en el select (solo falla el INSERT)
- ENSAMBLES (joins)
- Referencia una vista no actualizable en el FROM
- Subconsulta en el WHERE que referencia a una tabla en el FROM
- Referencia solo a valores literales only to literal values (sin tabla subyacente para actualizar)
- ALGORITHM = TEMPTABLE (*use of a temporary table always makes a view nonupdatable*)
- Múltiples referencias a cualquier columna de una tabla base (falla en INSERT, ok para UPDATE y DELETE)

Vistas Actualizables en MySQL (insert)

```
CREATE TABLE t1 (x INTEGER);  
CREATE TABLE t2 (c INTEGER);  
CREATE VIEW vmat AS SELECT SUM(x) AS s FROM t1;  
CREATE VIEW vup AS SELECT * FROM t2;  
CREATE VIEW vjoin AS SELECT * FROM vmat JOIN vup ON vmat.s=vup.c;
```

This statement is invalid because one component of the join view is nonupdatable:

```
INSERT INTO vjoin (c) VALUES (1);
```

This statement is valid; the view contains no materialized components:

```
INSERT INTO vup (c) VALUES (1);
```

Vistas Actualizables en MySQL (update)

This statement is valid; column `c` is from the updatable part of the join view:

```
UPDATE vjoin SET c=c+1;
```

This statement is invalid; column `x` is from the nonupdatable part:

```
UPDATE vjoin SET x=x+1;
```

This statement is valid; the updated table reference of the multiple-table UPDATE is an updatable view (`vup`):

```
UPDATE vup JOIN (SELECT SUM(x) AS s FROM t1) AS dt ON ...  
SET c=c+1;
```

This statement is invalid; it tries to update a materialized derived table:

```
UPDATE vup JOIN (SELECT SUM(x) AS s FROM t1) AS dt ON ...  
SET s=s+1;
```

Vistas Actualizables en MySQL (delete)

This statement is invalid because the view is a join view:

```
DELETE vjoin WHERE ...;
```

This statement is valid because the view is a merged (updatable) view:

```
DELETE vup WHERE ...;
```

This statement is valid because it deletes from a merged (updatable) view:

```
DELETE vup FROM vup JOIN (SELECT SUM(x) AS s FROM t1) AS dt ON ...;
```

Vistas Materializadas

- Algunas aplicaciones pueden requerir alto grado de respuesta por parte de la BD (no siendo suficiente la optimización de la/s consulta/s)
- Una vista puede ser **materializada**
 - el SGBD pre-calcula y almacena su contenido (físicamente, como sucede con una tabla)
 - Puede ser usada como si fuera una relación común (incluso crear índices para mejorar la performance de consultas)
 - Se genera duplicación de datos y necesidad de **mantener la consistencia** respecto del contenido de las tablas base
- Cuestiones a evaluar:
 - ¿Qué vistas deberían materializarse? (decisión compleja)
 - ¿Cómo debe ser la sincronización entre vistas materializadas y tablas base?
 - Aplicar actualización por regeneración de la vista? o incremental?
 - En forma inmediata (al darse algún cambio)? periódicamente? forzado?

Vistas - Ventajas

- **Simplifican la percepción** que los usuarios tienen de la BD, presentando la información necesaria y ocultando el resto
- **Presentan diferentes datos** a distinto tipo de usuarios, aún cuando los estén compartiendo (sobre la misma BD)
- **Permiten definir consultas complejas/frecuentes** para no tener que especificarlas cada vez que se utilizan
- **Facilitan la independencia de los datos** (ocultando a los usuarios cambios en la estructura en las tablas base)
- **Permiten aplicar políticas de seguridad** (privacidad): dando privilegios selectivamente sobre distintas vistas (control de acceso)
 - Vistas sobre algunas columnas, ocultando otras reservadas para usuarios específicos (ej. *antecedentes penales*)
 - Vistas sobre determinadas filas, ocultando otras reservadas para usuarios específicos (ej. *películas no aptas para público infantil*)



Vistas - Desventajas

- **Actualizaciones VISTA → Tabla BASE restringidas:** hay varias limitaciones sobre la estructura de las vistas para asegurar que resulten automáticamente actualizables (debido a posibles anomalías)
- **Cuestiones de Rendimiento:** el proceso de resolución de la vista puede exigir el acceso a múltiples tablas cada vez que se accede a ella → evaluar si podría justificarse su materialización (técnicas de mantenimiento de vistas)
- **Necesidad de sincronización** en caso de vistas materializadas (y duplicación de datos)
- Modificaciones a la estructura de tablas base (ej. agregado de columnas) no serán advertidos por la vista, salvo que sea re-creada



Vistas Materializadas

Sistema de administración de bases de datos

Cómo funcionan las vistas materializadas

PostgreSQL

Con PostgreSQL, se debe actualizar manualmente la vista materializada y volver a calcular la vista completa. La vista materializada se completa con datos en el momento exacto en que se crea.

MySQL

MySQL no admite las vistas materializadas.

Oracle

Oracle actualiza automáticamente las vistas materializadas, pero también ofrece la opción de actualizarlas a pedido. También puede escribir una instrucción SQL que solicite que las vistas se actualicen antes de generar resultados.

SQL Server

SQL Server usa el nombre "vistas indexadas", ya que la materialización es un paso para crear un índice de una vista normal. Solo puede realizar consultas SQL básicas con las vistas indexadas. Se actualizan automáticamente para el usuario.

Vistas Materializadas - PostgreSQL

```
CREATE MATERIALIZED VIEW view_name  
AS query  
WITH [NO] DATA;
```

- *view_name* is the name of your materialized view in Postgres
- *query* is that complex query that supplies the data for our materialized view
- *WITH DATA/ WITH NO DATA* is the parameter that specifies if the query has to load the data (the query results) at once during the object creation. If we need it while creating a Postgres materialized view, we specify the WITH DATA parameter. If not – use the WITH NO DATA one.

```
CREATE MATERIALIZED VIEW alumnos_2017  
AS select * from alumno where añoDeIngreso=2017  
WITH LOCAL CHECK OPTION  
WITH DATA;
```

BIBLIOGRAFÍA

Date, C., “An Introduction to Database Systems”. 8º ed., Addison Wesley, 2004

Elmasri, R., Navathe, S., “Fundamentals of Database Systems”, Addison Wesley, 2011 (Cap. 5)

Ramakrishnan R., Gehrke J., “Database Management Systems”, 3º ed. , McGraw-Hill, 2003 (Cap. 3 y 25)

Silberschatz, A., Korth, H, Sudarshan, S., “Database System Concepts”, McGraw Hill, 2001 (Cap. 4)